

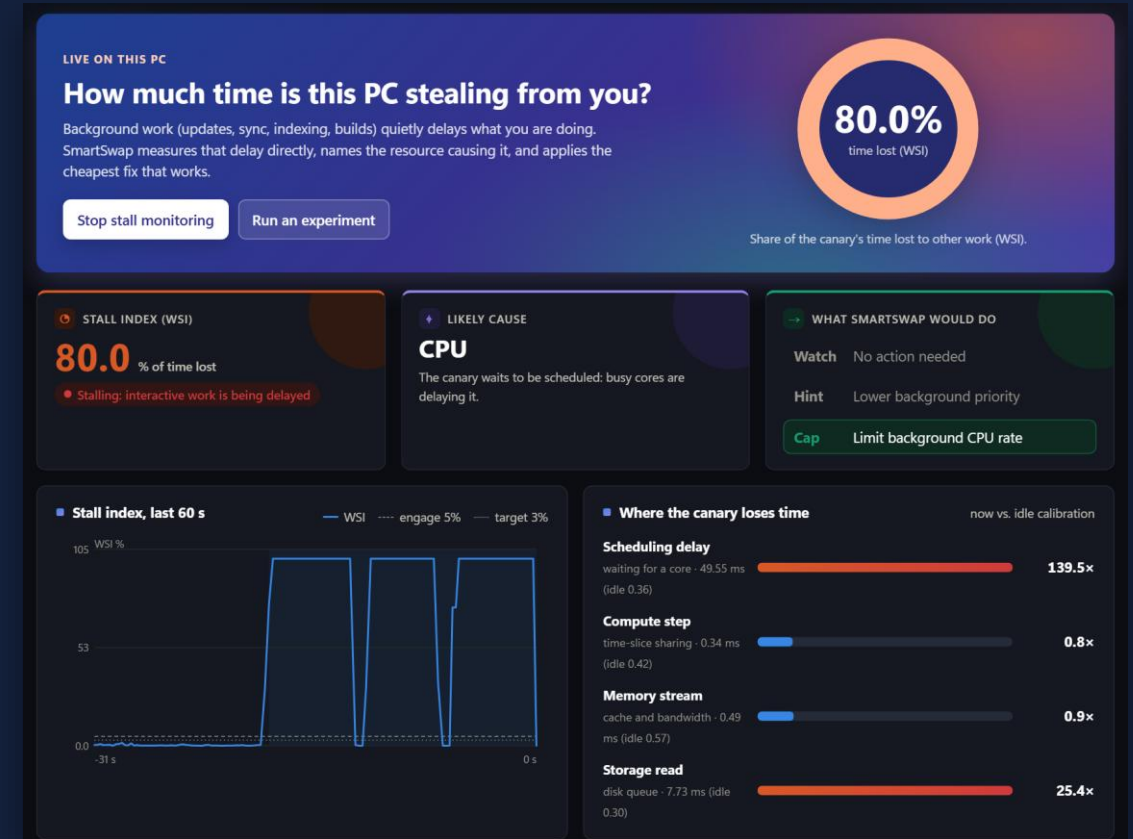
SmartSwap

Measuring, and fixing, the time background work steals from the app you are using on stock Windows

No admin rights
user-mode only

Names the cause
CPU · memory · storage

Proves every fix
read back from Windows



The live dashboard during real CPU contention: 80% of the canary's time lost, cause named, cap stage reached.

Task Manager shows how busy the PC is, not how much you are being slowed down

- 1 **Background work never stops.** Sync, search indexing, updates and builds run while you work. On our test laptop they used 3–4 cores without being asked.
- 2 **Busy is the wrong signal.** Memory-bandwidth contention can make an app several times slower while half the cores are idle. No Task Manager counter shows it.
- 3 **Today's fixes are blind.** Priority tweaks and “optimizer” tools change settings without measuring whether the user actually benefits.

8 of 16

CPU cores idle under memory-bandwidth load

yet

2.6× slower

interactive tail latency (p95 3.95 → 10.8 ms)

Problem statement Unprivileged software on Windows cannot measure how much time interactive work loses to background interference, tell which resource causes it, or verify that a mitigation helped.

Prior work measures stalls or controls interference, but not on an unprivileged desktop

Work	What it contributes	Why it does not solve our problem
Linux PSI (Weiner, 2018)	Kernel-reported share of time tasks stall on CPU, memory or I/O	Linux kernel only; Windows has no user-mode equivalent
Heracles (ISCA 2015), PARTIES (ASPLOS 2019)	Feedback controllers that co-locate latency-critical and batch jobs	Need server cache/bandwidth partitioning and privileged control
Caladan (OSDI 2020)	Reacts to interference within microseconds	Kernel-bypass runtime; the protected app must be rewritten
Bubble-Up, Bubble-Flux (MICRO 2011, ISCA 2013)	Synthetic probes measure how sensitive a job is to contention	Offline profiling for data-centre placement
Windows tools: Task Manager, priority, EcoQoS	Show utilisation; static deprioritisation	No harm measurement, no cause, no proof a change helped

Research gap No privilege-free way on Windows to measure interference harm, name its cause, and prove that a fix worked.

Five measurable objectives, all met in a 460-trial evaluation

- | | | |
|----|--|--|
| O1 | Measure interference harm from user mode, without admin rights | ✓ Stall index tracks the real app's p95: $\rho = 0.96$ |
| O2 | Identify which resource is contended | ✓ Cause named correctly in 86–99.6% of stalled periods |
| O3 | Apply the cheapest effective fix automatically | ✓ Tail latency cut by up to 94% |
| O4 | Verify every action and every revert | ✓ 1,575 / 1,575 actions · 322 / 322 reverts verified |
| O5 | Evaluate rigorously, including the cost to background work | ✓ Randomised blocks, exact Wilcoxon, Holm, bootstrap CIs |

Four ideas that no Windows tool combines today

1 Windows Stall Index

A tiny canary times its own delays to estimate how much time interactive work is losing: a PSI-style signal Windows lacks.

$\rho = 0.96$ with the real app's p95 · 1.2% false alarms

2 Specificity-ordered attribution

Checks the most specific symptom first: CPU, then memory, then storage. The naive “biggest slowdown” rule blames storage for memory contention.

Cause correct in 86–99.6% of stalled periods

3 Least-cost, double-verified ladder

Priority first, CPU cap only if needed. Every action is read back from Windows and kept only if the stall really drops.

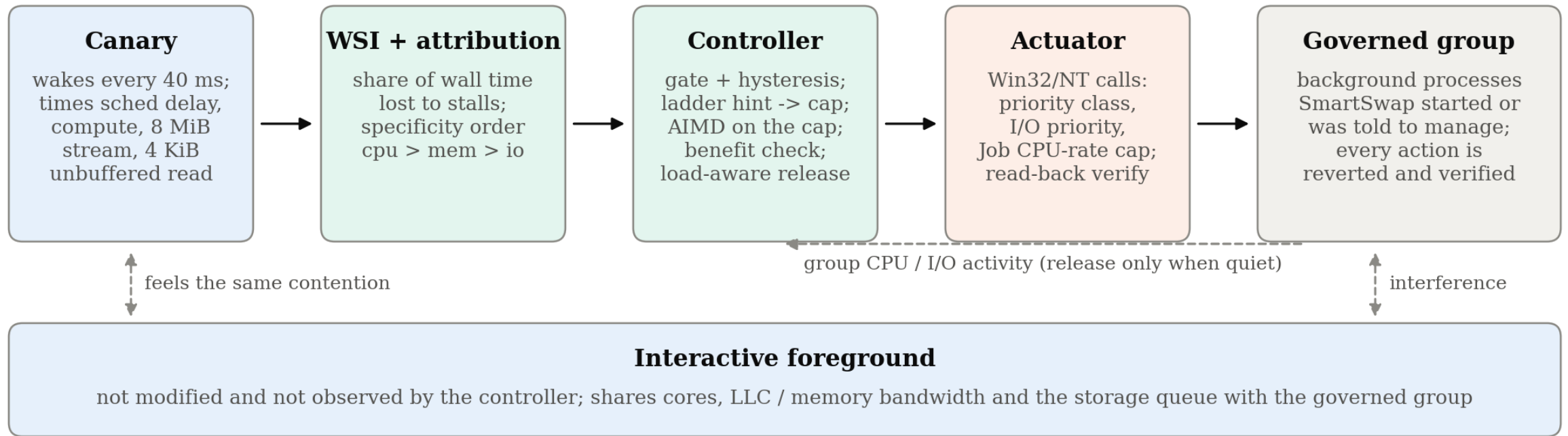
Without the ladder: 4.8× worse p95

4 Hybrid-core steering

On P-core/E-core laptops, moves background work onto efficiency cores before throttling it.

Memory contention: 27% lower p95 than the best static policy

A closed control loop that runs every 200 ms, entirely in user mode



Sense

Canary wakes every 40 ms and times scheduling delay, compute, an 8 MiB memory copy and a 4 KiB unbuffered read.

Decide

WSI = share of time lost. Cause = first symptom above 2× idle. Ladder level chosen against a 3% target.

Act & prove

Priority, I/O priority, E-core affinity or Job CPU cap via Win32/NT; read back, check the benefit, revert when quiet.

Six modules, each testable on its own

M1

✓ Complete

Canary & probes

`probes.py`

40 ms canary, foreground probe, CPU / memory / storage workloads

M2

✓ Complete

WSI & attribution

`controller.py`

Stall index and specificity-ordered cause detection

M3

✓ Complete

Control ladder

`controller.py`

Gating, hint → E-core steer → AIMD cap, benefit check, release

M4

✓ Complete

Verified actuator

`native.py`

Priority, I/O priority, affinity and Job CPU cap with read-back

M5

✓ Complete

Experiment engine

`engine.py · run_study.py`

Trials, randomised blocks, background-throughput accounting

M6

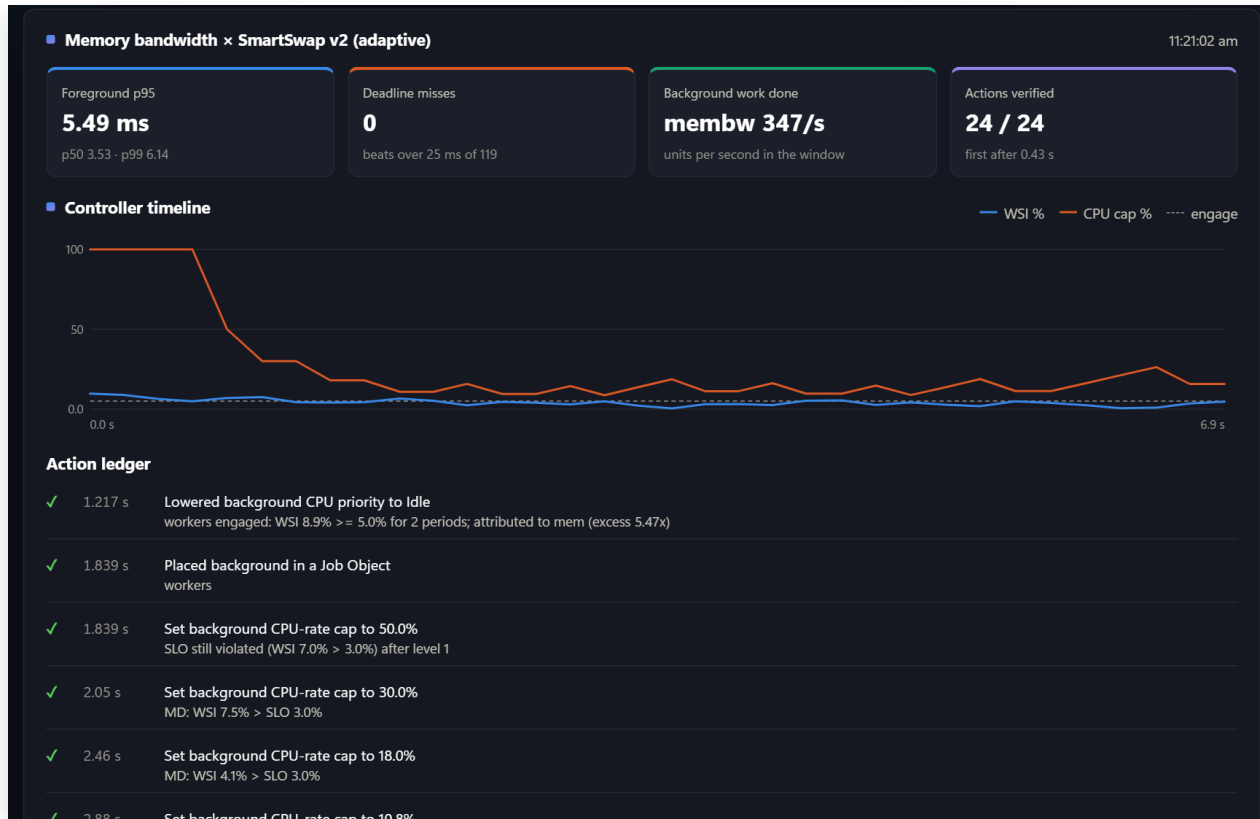
✓ Complete

Analysis & dashboard

`analyze.py · app.py · frontend`

Statistics, figures, live web UI and exports

All six modules are implemented, tested and verified on real hardware



Live trial from the dashboard: memory-bandwidth contention, p95 5.49 ms, 24 / 24 actions verified while AIMD tunes the cap.

6 / 6
modules complete

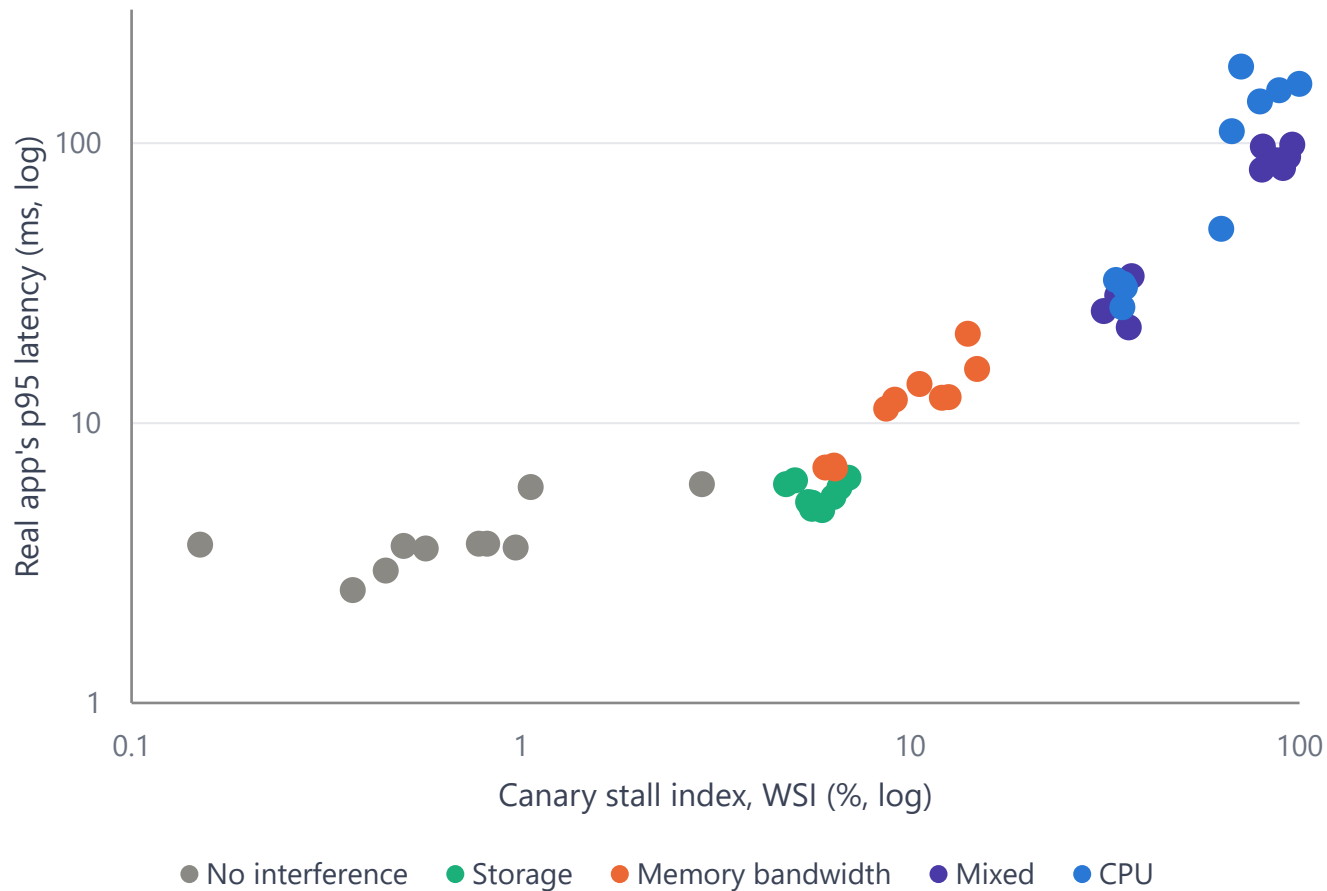
25
automated tests passing

460
trials run, 0 failed

1,575 / 1,575
actions verified by read-back

Also complete: live monitor page, CSV / JSON / HTML / PDF exports, emergency stop, and an IEEE-format paper draft.

The stall index tracks what the user actually feels



$\rho = 0.96$

rank correlation between the canary's WSI and the separate app's p95 (50 trials)

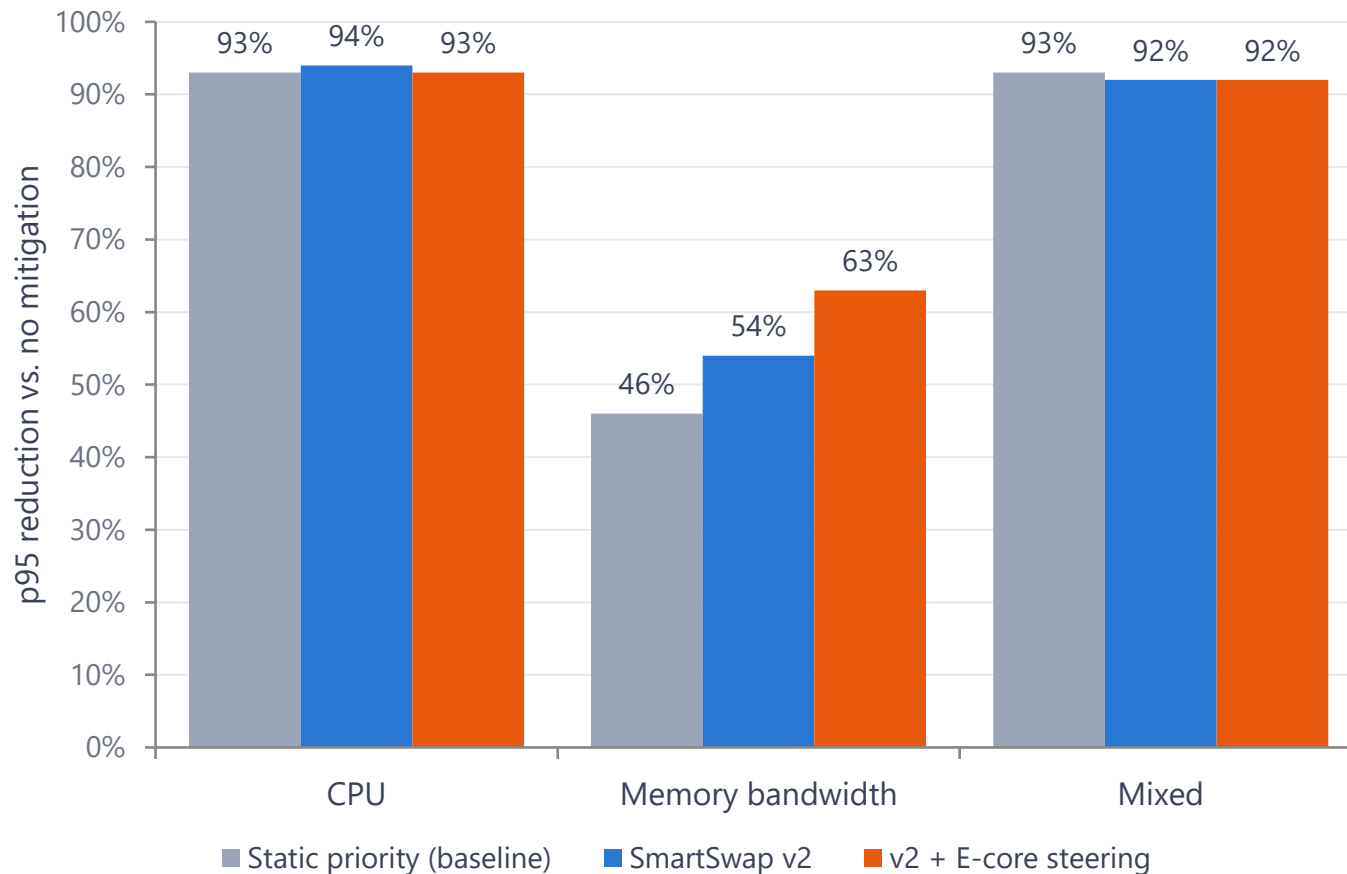
AUC 0.965–1.000

detecting interference, with 1.2% false alarms on a quiet PC

92 · 86 · 99.6%

cause named correctly for CPU · memory · storage

Up to 94% lower tail latency, and E-core steering beats static priority on memory contention



What we report honestly

- Static priority is a strong baseline; v2 matches it on CPU and mixed load.
- Under memory contention, v2 without steering over-throttles: 56% of background work kept vs 92% for priority.
- Steering is measured on one hybrid laptop (8 blocks); replication is next.

Review 3 plan

- IEEE-format paper drafted (8 pages)
- Replicate on a second Windows laptop
- Add a real app (browser frame time)
- Submit to an IEEE conference